Inputs, outputs, and readouts

ar087 · 14 August 2026 · Draft

Contents

Developer guide

Outputs and observables

An output is a named value returned by the graph. An observable exposes an internal signal for recording without changing the computation.

```
readout = snn.readouts.MeanVoltage(
    source=cell.E.spikes,
    classes=10,
    name="classifier",
)
net.output("class_logits", readout)
net.expose(cell.E.spikes, cell.I.spikes, name="hidden")
```

Standard readouts

The standard readouts are mean voltage, final voltage, spike count, duration-normalized spike rate, and cumulative potential. Spike-rate logits divide spike count by valid presentation duration, so changing duration does not silently rescale the classifier.

All five standard readouts execute in the graph executor. Spike rate reports spikes per second from either an explicit duration in seconds or a `(time, batch)` valid-time mask; masked reductions reject empty windows rather than silently returning arbitrary values. The compiler infers shapes and units, checks linear readout parameter shapes, rejects malformed masks and ambiguous durations, and records operation requirements in the bundle capability manifest.

Input bindings

The execution layer binds concrete data to graph inputs without placing datasets or stimuli in graph structure. Dense-array bindings are implemented as named, data-only values. NPY replay binds to a graph with one input; NPZ arrays bind by input identifier.

Before execution, the resolver requires exact input coverage, common time and batch axes, declared feature dimensions, finite numeric values, binary spike values, and boolean or zero/one masks. A mismatch names the offending input and fails before simulation.

Every resolved dense replay emits a versioned execution protocol containing the representation, source-file digest and array key, resolved shape and data type, signal type and unit, dataset identity and split when supplied, sample cap, batch size, shuffle behavior, timestep, duration, masks, and execution seed. The command-line contract accepts `--input-file`, `--input-dataset-id`, `--input-split`, and the explicit `--input-shuffle` or `--no-input-shuffle` pair. The typed request accepts `DenseArrayBinding` values; the original in-memory tensor mapping passes through the same resolver.

Fixed-rate and categorical variable-rate Poisson encoding belong to execution protocol rather than graph topology. Dense arrays and event streams need separate bindings because their storage and timing semantics differ.

Dense-array and valid-time-mask bindings work today. Event-stream, portable dataset-loader, and encoder bindings are not implemented.

API reference

Outputs and observables

```
net.output(name, signal) -> SignalLike
net.expose(*signals, name=None) -> None
```

An output maps a public name to one graph signal. `expose` records one or more internal signals. A single exposed signal uses `name` directly; multiple signals use `<name>_0`, `<name>_1`, and subsequent indices.

Standard readouts

```
snn.readouts.MeanVoltage(*, source, classes, name, tau=20 * snn.ms,
weight=Normal(1.0, 0.1))
snn.readouts.FinalVoltage(*, source, classes, name)
snn.readouts.SpikeCount(*, source, classes, name)
snn.readouts.SpikeRate(*, source, classes, name, duration=None, mask=None,
window="full")
snn.readouts.CumulativePotential(*, source, classes, name)
```

Every constructor returns a `Readout`, which delegates signal attributes and exposes its parameter identifiers. `classes` is the output width. `SpikeRate` requires either `duration` in seconds or a `(time, batch)` mask. The only implemented window is `"full"`.

Dense bindings

```
DenseArrayBinding(input_id, value, source={})
load_dense_array_bindings(path, graph) -> tuple[DenseArrayBinding, ...]
resolve_dense_array_bindings(
    graph, *, bindings=(), inputs=None,
    device="cpu", seed=0, protocol=None,
) -> ResolvedDenseInputs
```

`value` is a PyTorch tensor. `source` is JSON-serializable provenance. NPY binds only to a graph with one input; NPZ keys bind by graph input identifier. Direct `ExecutionSpec.inputs` values are converted to in-memory bindings and validated through the same resolver.

The resolved protocol uses schemas `tools/snn.dense-array-binding/v1` and `tools/snn.execution-protocol/v1`. Reserved protocol fields cannot be overridden by caller metadata.

Next: Training recipes and graph-native learning